Project Proposal

Team C4

1 Team members

Muhammad Hamas
Muhammad Ali
Moeez Mufti
Talha Khan

2 Introduction

The proposed project applies IoT and embedded systems concepts to the domain of automated industrial quality control (QC). The system combines machine vision, distributed embedded controllers, sensors, actuators, and wireless communication to automatically image a placed object from multiple viewing angles and classify it as pass or fail.

The target application is an automated visual-inspection prototype: a "QC station" — capable of inspecting a mechanical part placed on a motorized turntable. When an operator places a part and presses a start button, the part is rotated to a fixed set of angles and photographed at each one, producing a consistent multi-view image set for that inspection. Each image set is stored per inspection and is intended to be evaluated by an edge-AI classifier that returns a pass/fail judgement. The system simulates an end-of-line inspection cell in a mechanical workshop, replacing slow and inconsistent manual visual checking with a repeatable, automated capture-and-classify cycle.

3 Concept description

The main purpose of the prototype is to demonstrate:

- Distributed embedded systems
- Industrial automation and automated visual inspection
- Real-time machine vision
- Embedded communication protocols (MQTT and HTTP)
- Automated object classification (pass/fail quality control)

The proposed system intends to combine embedded Linux computing, lightweight edge-AI inference, real-time embedded control, wireless communication, and industrial automation concepts to create a machine-vision-based quality-control station.

A Raspberry Pi 5 acts as the central coordinator. It runs a Flask application that owns the inspection state machine, drives the capture sequence, receives and stores images, and hosts the local MQTT broker. It is also the intended host for image classification using OpenCV and TensorFlow Lite.

An ESP32-CAM performs image acquisition on demand, capturing a single JPEG per view and uploading it to the Pi.

An Arduino Uno WiFi Rev2 handles low-level actuator control, driving the servo that indexes the turntable to each fixed viewing angle and reporting back the angle it reached so the coordinator can verify correct positioning before an image is taken.

Communication is split by payload type to match each transport's strengths. Small, latencysensitive control and event messages between the Pi and the camera use MQTT over Wi-Fi at QoS 1, chosen for its lightweight, low-latency, publish/subscribe characteristics that suit distributed embedded systems. Turntable control, the operator start trigger, and the comparatively heavy JPEG image payloads use HTTP over Wi-Fi, keeping bulk image transfer off the messaging bus. This division gives a clear, inspectable communication architecture in which command flow and image flow are decoupled.
Two components are identified as planned extensions and are intentionally scoped out of the initial build: the edge-AI classifier on the Raspberry Pi (which will populate the reserved per-image classification and confidence fields), and a visual dashboard for live inspection feedback. The current data model and API already reserve space for both so they can be integrated without disturbing the core capture pipeline.

4 Project/Team management

Team Member	*Tasks (planned)*
Muhammad Hamas	*AI Model Training and system planning*
Muhammad Ali	*Dashboard creation via Flask and MQTT client integration for data visualization*
Moeez Mufti	*Hardware setup, Arduino Uno Rev2/ESP32CAM WiFi setup and documentation*
Talha Khan	*MQTT broker setup on Raspberry Pi and establishing communication between Arduino Uno Rev2, ESP32CAM & Raspberry Pi*

5 Technologies

Devices

- Raspberry Pi 5 — central coordinator, image storage, and edge inference host
- ESP32-CAM (AI-Thinker) — image-capture node
- Arduino Uno WiFi Rev2 — turntable controller

Sensors

- Camera sensor (ESP32-CAM / OV2640) for image capture
- Operator push-button as the inspection start trigger
- Positional (angle-indexed) servo used to place the turntable at fixed, repeatable viewing positions

Actuators

- Servo motor driving the turntable to fixed indexing angles (0°, 60°, 120°, 180°)

Communication

- MQTT over Wi-Fi (QoS 1) — camera capture commands, camera acknowledgement/result events, and camera status/heartbeat messages, all under the topic root qc/camera/qc-camera-01/
- HTTP over Wi-Fi — turntable control commands (Pi → Arduino), the inspection start trigger (Arduino → Pi), and raw JPEG image upload (ESP32-CAM → Pi)
- Mosquitto MQTT broker hosted locally on the Raspberry Pi

Software and Tools

- Python — coordinator service (Flask) and MQTT client (paho-mqtt)
- C/C++ — ESP32-CAM and Arduino firmware, servo and capture control
- OpenCV — image handling and pre-processing
- TensorFlow Lite — edge classifier for defect / pass-fail inference (planned)
- Arduino IDE
- Mosquitto MQTT Broker

6 Sources/References

https://github.com/MAliSohail/Team-C4-Advanced-Embedded-Systems-Lab